CS-301 · Operating Systems · Semester 5

Page Replacement Algorithms: FIFO, LRU and Optimal

Prepared by Ayesha Siddiqui

1. Why page replacement is needed

With demand paging, a page is loaded into a physical frame only when it is first referenced. When a process references a page that is not in memory, a page fault occurs and the operating system must load it from disk. If no frame is free, the OS chooses a victim page to evict. The page replacement algorithm decides which page that victim is.

The goal is to minimise the number of page faults, because each fault costs a disk access, which is roughly a million times slower than a memory access. A dirty bit records whether the victim was modified; only dirty pages must be written back to disk before the frame is reused.

2. FIFO and Belady's anomaly

FIFO evicts the page that has been in memory the longest. It is simple to implement with a queue but ignores how often or how recently a page is used.

FIFO suffers from Belady's anomaly: adding more frames can increase the number of page faults. For the reference string 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5, FIFO causes 9 page faults with 3 frames but 10 page faults with 4 frames. Stack algorithms such as LRU and Optimal never show this anomaly.

3. Optimal (OPT)

The optimal algorithm evicts the page that will not be used for the longest time in the future. It gives the lowest possible fault rate for a fixed number of frames, but it cannot be implemented in practice because the OS does not know future references. It is used as a benchmark to judge other algorithms.

4. Least Recently Used (LRU)

LRU evicts the page that has not been used for the longest time in the past, using recent history as a prediction of the future. It is implemented with a counter (timestamp) per page or with a stack of page numbers where the most recently used page is moved to the top.

Worked example: reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2 with 3 frames. FIFO causes 10 page faults, LRU causes 9 page faults, and Optimal causes 7 page faults.

Because exact LRU needs hardware support on every memory reference, real systems use approximations such as the Clock (second-chance) algorithm, which keeps a reference bit per frame and a circular pointer; a page with reference bit 1

gets a second chance and its bit is cleared.

5. Thrashing

Thrashing happens when a process does not have enough frames for its working set, so it spends more time paging than executing. CPU utilisation drops, and a naive scheduler that admits more processes to raise utilisation makes it worse. The working-set model and page-fault-frequency control prevent thrashing by giving each process enough frames for its current locality.